Master's Thesis Computing Science

Radboud University Nijmegen

Evaluating Data Stream Write Performance in Current Lakehouse Systems

Author:
Vinícius Ribeiro Machado Schmidt
s1123702

University supervisor:
prof. dr. ir. Djoerd Hiemstra
hiemstra@cs.ru.nl

Company supervisor:
Chrystel Philipsen
chrystel.philipsen@sogeti.com
Marcel de Wit
marcel.de.wit@sogeti.com

Second assessor:
dr. Johannes Textor
johannes.textor@ru.nl

January 28, 2026

Abstract

Contents

Chapter 1

Introduction

1.1 Motivation and research questions

Many modern data storage architectures rely on so-called *data lakes*, which consist of cloud object stores, e.g. Amazon S3[1], Azure Blob Storage[2], for storing large amounts of (un)structured and semi-structured data in a centralized place [33]. Part of the data can then be copied into data warehouses through a process called "extract, transform, load" (ETL), so that it can be used for further downstream analytics tasks [22]. Although this two-tier architecture has become widespread among many enterprises, it has a few drawbacks. ETL pipelines used to load data into warehouses can be time-consuming and error-prone, causing the data in the warehouse to be outdated and sometimes inconsistent [26]. Additionally, maintaining the two systems is expensive and redundant, since part of the data is stored in both the data lake and the warehouse [22]. Finally, the flexibility of storing various file formats in the same place comes with the cost of managing different file versions and metadata, which with time can transform the data lake into a "data swamp" [5].

Following the limitations of data lakes and the two-tier architecture, Michael Armbrust et al. proposed a new data storage architecture called *data lakehouse*. A lakehouse, also referred to as an open table format, builds on top of data lakes by structuring data into tables that are stored in immutable columnar formats, e.g. Apache Parquet[3]. These tables are managed with centralized metadata files that keep track of the current schema and the different versions of the table, all while ensuring that updates to tables occur in ACID [12] transactions [22]. Furthermore, the use of open formats for storage gives users control over their data, in contrast to proprietary formats of data warehouses [5], [22], [26]. These characteristics allow analytical tasks

[1] `https://aws.amazon.com/s3/`
[2] `https://azure.microsoft.com/en-us/products/storage/blobs/`
[3] `https://parquet.apache.org/`

to be performed directly on the lakehouse tables, thus eliminating the need of having a separate data warehouse, thus eliminating the need for a two-tier architecture.

Although open table formats have gained popularity since their introduction, some aspects of lakehouses have led to important questions in recent works. For instance, Paras Jain et al. suggest that performing high frequent insertions into lakehouse tables can be challenging, because every insertion creates a new data file and requires updating the metadata files as well. Further, inserts in Delta Lake, a popular lakehouse format, can have up to hundreds of milliseconds of latency due to its reliance on object stores, which limits the throughput of streaming workloads, as mentioned in [4]. By contrast, the recent introduction of DuckLake [21] brought significant changes on how to handle write transactions to tables, while still ensuring ACID transactions. This is all thanks to the use of a traditional database management system (DBMS), e.g. PostgreSQL[4], for handling table metadata. Interestingly, no work has been done yet to evaluate the throughput of open table formats when performing data stream insertions. Therefore, this thesis will fill that gap, by focusing on the following research question:

"*How can we develop a benchmark to evaluate the throughput of data streams in lakehouse systems?*"

In addition, the following sub-questions will be answered

1. What are the current state-of-the-art benchmarks for lakehouses?
2. What are common patterns used in benchmarks for stream processing systems?
3. How can we ensure our benchmark is actually fair?
4. What is the difference in performance of current lakehouse systems when evaluated with our benchmark?
   (a) What is the performance gain of the "data inlining" feature of DuckLake in comparison to other lakehouse formats?

1.2 Contributions

1.3 Thesis structure

The rest of this thesis is structured as follows: Chapter 2 goes over the relevant background for this thesis, such as the different lakehouse technologies, how they are implemented and relevant features for this research. Chapter 3

[4] `https://www.postgresql.org/`

talks about the existing work on benchmarking lakehouses and the different metrics employed by these benchmarks and the systems that they evaluate. Chapter 4 describes the methodology employed to develop our own benchmark and the reasoning behind those decisions. We also describe our experimental setup here. Chapter 6 present the results of our experiments, which will be discussed in Chapter 7. Finally, Chapter 8 will conclude this thesis by looking back at the proposed research questions, while proposing possible directions for future work.

Chapter 2

Background

Before we can develop our benchmark, we need to first understand how lakehouses work and which characteristics are relevant to the goal of this thesis. We will look at Iceberg and Delta Lake (Sections 2.1 and 2.2), which are the two most popular and more mature lakehouse formats based on GitHub stars count and initial release date respectively [17], [30]. Moreover, we will look at DuckLake, which brings innovative and relevant features for our research (discussed in Section 2.3), despite it still being in version 0.3.

This chapter will also go over the definitions of data stream and throughput, as well as guidelines on performing fair benchmarking.

2.1 Iceberg

Iceberg was developed in 2017 at Netflix and in 2018 it was released as an open-source project within the Apache Software Foundation[1].

2.1.1 Architectural design

Data layer

At the base of Iceberg's architecture, there is a data layer, which consists of data files that constitute the Iceberg table. They can be stored in one of three formats: Parquet, ORC[2] or Avro[3], with Parquet being the most common due to its widespread adoption and performance gains [19].

Metadata layer

Besides the data layer, there is the metadata layer, which keeps track of the data files locations, the version history of the table, as well as its current

[1] `https://www.apache.org/`
[2] `https://orc.apache.org/`
[3] `https://avro.apache.org/`

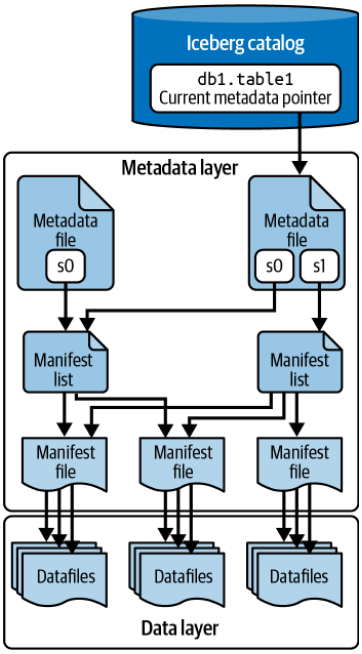

Figure 2.1: Architecture of an Iceberg table, as presented in [19].

schema. The files in the metadata layer are:

- **Manifest files**: Stored in Avro format, these files store pointers to multiple data files, as well as metadata about them, such as the minimum and maximum values of a file's columns and the number of records [19]. The metadata in manifest files is used for filtering data files during read operations, which helps improve read performance [19].
- **Manifest lists**: Manifest lists are another set of Avro files that individually point to a collection of manifest files. Together, the manifest files referenced by a manifest list constitute a specific snapshot, i.e. version, of an Iceberg table [19].
- **Metadata files**: These are JSON files that contain schema information about the table, as well as the snapshot history and a pointer to the current snapshot of that table [19]. A new metadata file is created after every update to the table, marking the creation of a new snapshot. Figure 2.1 depicts the architecture of an Iceberg table, where the newest metadata file has a pointer to both snapshots "s0" and "s1".

Catalog

In order to manage Iceberg tables, a catalog server is used. The goal of the catalog is to point to the latest metadata file of a table, as well as ensure that

this pointer is updated atomically [18], [19]. Consequently, all interactions with an Iceberg table must go through the catalog server. This avoids inconsistencies when clients are concurrently writing to the same table.

Although many technologies can be used as catalog, e.g. AWS Glue[4] and Hive Metastore[5], the developers of Iceberg provide a REST API spec [3] as an attempt to unify catalog access across different engines and languages. One prominent implementation of the Iceberg REST catalog is Apache Polaris[6], which uses a PostgreSQL database to store the latest metadata pointer.

2.1.2 Writing to Iceberg tables

Writing data to an Iceberg table, e.g. via `INSERT`, involves the following steps [19]:

1. First, the engine checks the catalog to get a pointer to the current metadata file, so that it can read the current table schema.

2. New data file(s) are written using the chosen storage format for the table.

3. The engine will then write the manifest file(s) that will contain metadata about the new data file(s), as well as a pointer to them.

4. Next, a new manifest list file is created, which will point to the new manifest file(s). In addition, existing manifest files are added to this new list, which together will correspond to the new snapshot of the table.

5. The engine writes a new metadata file which contains a pointer to the new snapshot (manifest list), as well as a history of all previous snapshots.

6. Finally, following the principle of optimistic concurrency [18], the engine will assume the table has not changed since the start of the operation and will try to atomically update the catalog to point to the new metadata file. However, if a concurrent write has created a new snapshot in the meantime, changing the metadata pointer fails and the engine has to retry. Iceberg tries to structure changes in such a way to minimize the cost of retries [2].

[4] `https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html`
[5] `https://hive.apache.org/`
[6] `https://polaris.apache.org/`

2.2 Delta Lake

Delta Lake was introduced in 2020 by Armbrust et al. It was originally developed at Databricks[7] and it was open-sourced in 2022 [29].

2.2.1 Architectural design

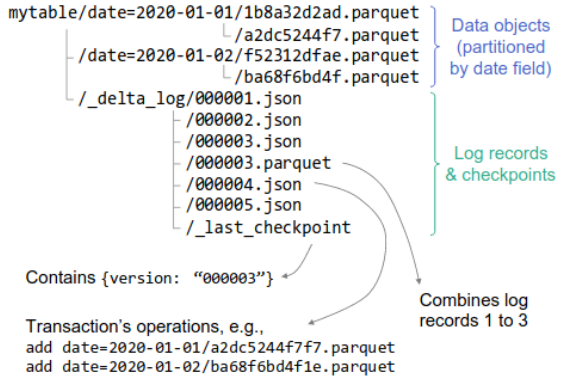

Figure 2.2: The architecture of a Delta Lake table, as presented in [4].

Data files

At its core, a Delta table consists of a directory containing data files, which are stored in the Parquet format. As shown if Figure 2.2, this data can be partitioned to improve efficiency of read queries, but this is beyond the scope of this thesis.

Delta log

The metadata of a Delta table is stored in a subdirectory, called "delta log". This directory contains JSON files, each named in increasing order according to the table version they correspond to, e.g. `00001.json`, `00002.json`, `00003.json`, etc. The log files themselves consist of actions, e.g. `add` and `remove`, that were applied to the data files in order to obtain the respective version [4].

Furthermore, Delta also keeps track of so-called "checkpoint" files. These are Parquet files consisting of all the actions in the JSON files up until that point in time. These checkpoint files are created periodically (the default being every 10 transactions), so that Delta can stay performant when reconstructing the current state of a table from the log files [4]. The name of the most recent checkpoint file is stored in a `_last_checkpoint` file, as depicted in Figure 2.2.

[7] `https://www.databricks.com/`

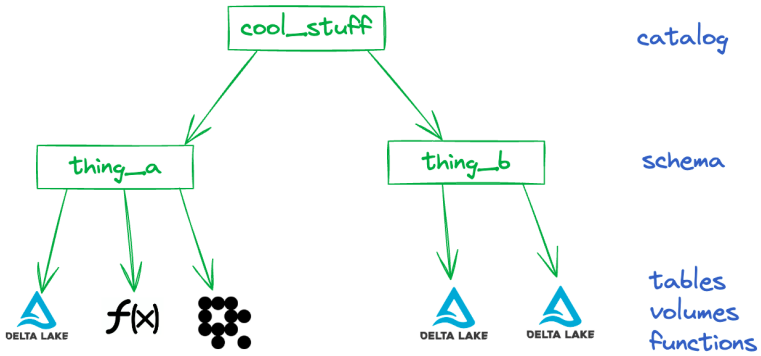

Figure 2.3: Overview of a Unity Catalog, which can be used to manage multiple Delta Lake tables [32].

Catalog

Delta Lake can be connected with a catalog, allowing the management of multiple tables and creating interoperability between other open table formats, such as Iceberg. The most popular catalog used to manage Delta tables is Unity Catalog[8]. There are two versions of this catalog: A proprietary version maintained by Databricks and an open-source version. We focus on the latter, since it is freely available. Similar to the Polaris catalog, the Unity Catalog server can be accessed through a REST API and it uses a relational database to store information about the Delta tables that it knows about.

Although a catalog server is not part of the Delta Lake specification, as it is in Iceberg's case, we include it in our research to make sure we perform a fair comparison between the systems we are analyzing (see also Section 2.5).

2.2.2 Writing to Delta tables

The process of writing data to a Delta table can be broken down into the following steps [4]:

1. Delta will first look for the `_last_checkpoint` file. Using the checkpoint file's ID, all subsequent log and checkpoint files are listed.

2. Using the listed files, the engine will read them to reconstruct the current state of the table, as well as statistics such as min and max values of columns and partitioning information.

3. Once all those files have been read, new data objects are written (possibly in parallel) to the underlying file storage.

[8] `https://www.unitycatalog.io/`

4. Then, assuming the latest log file has ID n, Delta attempts to create a new log file named $n + 1$`.json`. Since there can only be one file corresponding to a specific version of the table, creating the new log file needs to happen atomically. If this step fails due to a concurrent write operation, the transaction can be retried.

5. (Optional) Depending on the configured interval for writing checkpoints, a new checkpoint file will be created and the `_last_checkpoint` file is updated to reflect that change.

It is important to note that, like in Iceberg (Section 2.1.2), optimistic concurrency control is employed for write operations. However, Delta relies on the underlying object store's mechanism to ensure transaction atomicity [4]. For instance, Azure Blob Store, Google Cloud Storage and Amazon S3 support conditional write operations [16], [25], [27], thus ensuring that only one client can create a log file with a specific name.

2.3 DuckLake

DuckLake is an open table format introduced in 2025 as part of the DuckDB Foundation. Although it is currently an experimental release (as of writing, the current version is 0.3 [11]), we believe it will be a valuable addition to our research, as its specification proposes a new way of storing metadata and an interesting approach for handling changes to its tables.

2.3.1 Architectural design

Data layer

DuckLake's data layer consists of Parquet data files on the chosen storage system, which make up the table.

Catalog database

Similar to Iceberg, DuckLake also includes a catalog in its specification. However, DuckLake's catalog differs from Iceberg's in that it stores all of the table's metadata instead of storing the pointer to the current metadata file. In total, the catalog database contains 22 tables that together keep track of the schema, snapshots, data files, tables, statistics and partitioning information of DuckLake tables [9]. Currently, DuckLake supports DuckDB[9], SQLite[10], PostgreSQL and MySQL[11] as its catalog database [6]. Figure 2.4 shows the most relevant tables stored in the catalog [9]:

[9] `https://duckdb.org/`
[10] `https://sqlite.org/index.html`
[11] `https://www.mysql.com/`

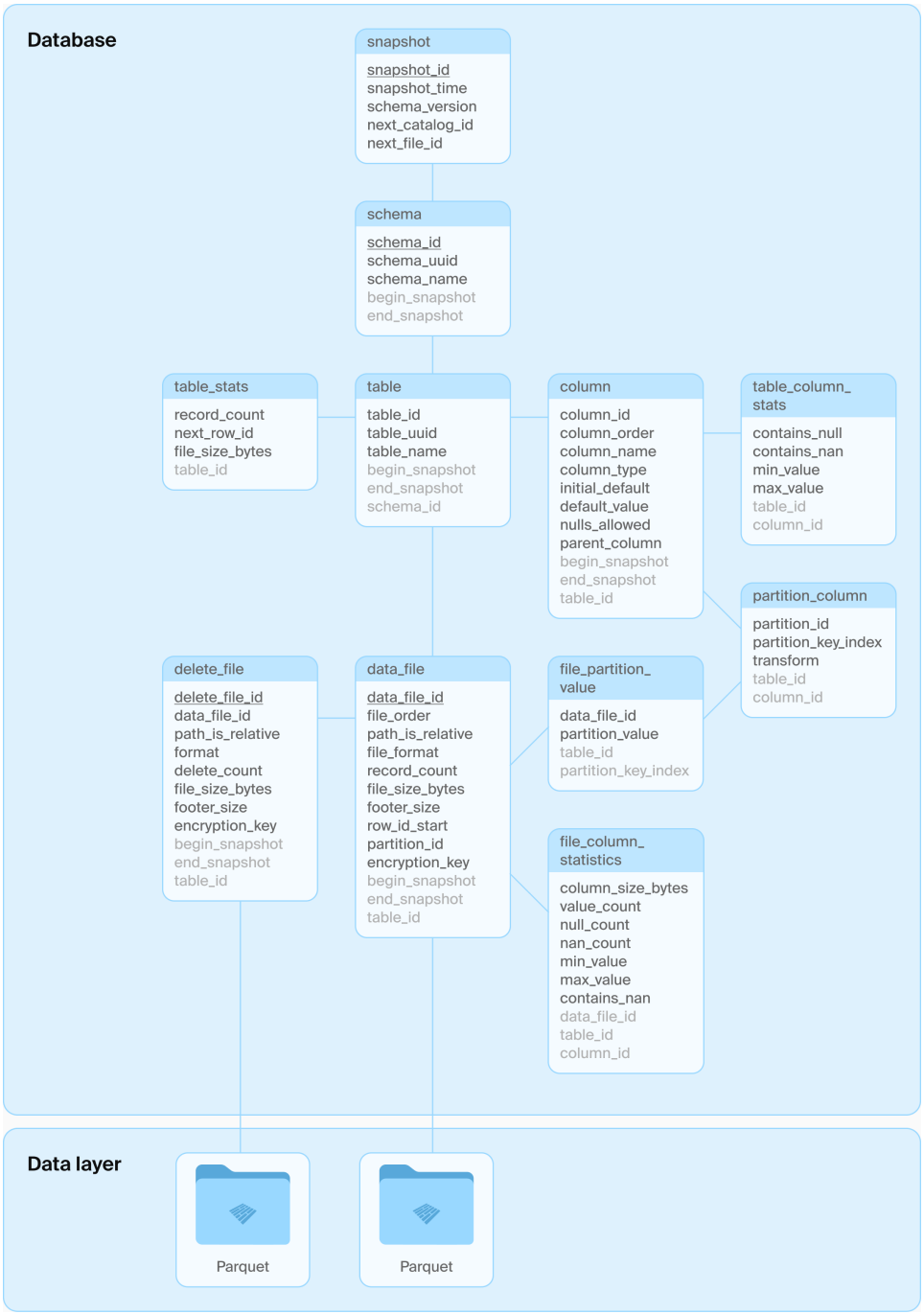

Figure 2.4: Architecture of a DuckLake table, as presented in [21].

- `table`: This is where the DuckLake tables' name, path and schema are stored. In addition, it tracks the snapshot interval in which the table is valid.
- `table_stats`: This contains metadata about the DuckLake tables, such as the number of records and the total file size of the data files that compose this table.
- `column`: This table stores the columns that are part of the DuckLake table. The main attributes are the column's name, its type and default value.
- `table_column_stats`: Similar to `table_stats`, this table contains metadata about each column in a DuckLake table, e.g. min and max values and whether the column has `NULL` values.
- `partition_column`: Contains partitioning information about columns on which partitioning has been enabled. It also stores the transformation function that is applied to the column to obtain the partition key, if such a function is required. For instance, if a table contains a timestamp column, one can apply a transformation function to that column and use the resulting value as the partitioning key.
- `data_file`: This table stores information about the data files, such as the path, format (for now, only Parquet is allowed [9]), number of records in the file and the size in bytes.
- `file_column_stats`: Similar to `table_column_stats`, this table stores column statistics on a file level, e.g. amount of records, min and max values and number of `NULL` values.
- `delete_file`: Similar to `data_file`, this table stores the path and format of delete files[12], but it contains the number of records deleted by the file.
- `snapshot`: This table is used for representing the snapshots, i.e. versions, of the DuckLake instance. Every change to a table will result in a new snapshot with a corresponding timestamp.
- `snapshot_changes`: For each snapshot, this table stores a list of changes that were made to the DuckLake instance, e.g. `CREATE`, `INSERT`, `UPDATE`, `ALTER`.

[12]Delete files are used to mark which records have been deleted from a table without actually deleting any files. This is done because Parquet files are immutable and would have to be rewritten every time a record is updated or deleted. By using delete files, deleted records can be reconciled at read time in a process called merge-on-read (MoW) [23], which is beyond the scope of this thesis.

- `schema`: This table stores the different schemas that can be used to separate DuckLake tables into namespaces. The table stores the schema name, its path in the file system, as well as the snapshot interval in which that schema is valid.

2.3.2 Writing to DuckLake

Writing to DuckLake tables happens in the following steps [21]:

1. First, the Parquet files are written to storage, unless data inlining (2.3.3) is enabled and the number of inserted rows is lower than the set threshold.

2. Then, in a single database transaction, the following tables are updated to reflect the new changes:

   - `data_file`
   - `table_stats`
   - `table_column_stats`
   - `file_column_statistics`
   - `snapshot`
   - `snapshot_changes`

   DuckLake enforces a primary key constraint on the snapshot id in the `snapshot` table [7]. If two clients try to concurrently create a new snapshot, one of them will fail due to a `PRIMARY KEY` constraint violation [7]. In that case, DuckLake will try to solve the conflict without having to rewrite the data files. For that, it looks at the `snapshot_changes` table to see what changes were made by the transaction that succeeded. If there are no logical conflicts, e.g. both clients just added new data, the failed client can retry the transaction without having to rewrite the data files [7].

2.3.3 Data inlining

One feature that sets DuckLake apart from Delta and Iceberg is the so-called "data inlining". This feature allows DuckLake to be configured such that writes that insert fewer rows than a set threshold are stored in the catalog database, rather than written out as Parquet files [8]. This can increase the performance of DuckLake for frequent, small, updates. The "inlined" data is immediately visible for read operations and it can be flushed out to Parquet files once enough data has been accumulated in the catalog [8].

2.4 Data streams

A data stream can be defined as data that is produced by one or more sources continuously and thus, is considered unbounded [10], [31]. Examples of data streams are measurements produced by IoT sensors, web server logs and network traffic data. Another key characteristic of stream data is the presence of at least one timestamp, which is assigned when the data point, i.e. event, is produced or processed by a system [10].

2.4.1 Throughput

When benchmarking streaming processing engines (SPEs), a common metric used is the system's throughput [14], [20], [28]. This metric can be defined as the amount of events that the system under test (SUT) can process in a given unit of time [20], [28].

2.5 Benchmarking

When it comes to evaluating systems' performance, it is crucial that the comparison is done fairly to avoid misleading results. In that regard, Raasveldt et al. highlight some of common pitfalls when benchmarking database systems. These pitfalls were obtained from a literature review on best practices and recommendations for both benchmarking in general and benchmarking of database management systems (DBMSs). The authors then identified seven[13] different pitfalls that can occur during DBMS benchmarking, namely:

1. **Reporting non-reproducible results**: In scientific research in general, it is required that results are reproducible, so that they can be independently verified. Benchmarking is no exception to this rule.

2. **Sub-optimally optimizing systems**: In cases where the authors of a paper are comparing their own system with the state of the art, they might feel tempted to not properly optimize the systems they are comparing with, so that they get favorable results [24]. In some other cases, failure to properly optimize a system might happen due to lack of familiarity with that system.

3. **Overly-optimizing a system**: Similarly to the previous pitfall, one might optimize their system to perform really well on a specific benchmark. This can be done by optimizing a system for the specific dataset or workloads present in a benchmark, which are all known prior to execution.

[13] In the original paper, the authors report "cold" vs "hot" runs and "cold" vs "warm" runs separately [24] , so they reported eight pitfalls in total. We believe however that they pertain to the same aspect of benchmarking, so we include them all in the same pitfall.

4. **Benchmarking code that contains bugs**: Sometimes, performance measurements can be influenced by bugs in the SUTs or in the code performing the benchmark. Bugs can lead to an unfair advantage by skipping a step of the benchmark workload or by only working with a specific dataset [24].

5. **Comparing systems that are fundamentally different**: Benchmark results can only be considered fair if they were obtained from systems that provide the same functionality. Otherwise, one of the systems might be given an unfair advantage by avoiding a set of constraints present in other systems. Raasveldt et al. exemplify this by comparing a complete DBMS with an algorithm specialized in one task. The algorithm has an advantage because it takes fewer aspects into account than the DBMS [24].

6. **Ignoring preprocessing time**: When running benchmarks, the SUTs often have to be initialized with the dataset used in benchmark's workload. This initialization can sometimes include preprocessing steps that a system executes to speed up performance of subsequent tasks. The example given in [24] mentions the creation of database indexes, which if ignored, might favor a system that has efficient, but slow-to-create, indexes.

7. **Comparing "cold", "warm" and "hot" runs**: The concept of a "cold" run refers to executing a benchmark for the first time on a system for which no data has been cached on the host operating system yet. By contrast, subsequent runs of the benchmark are considered "hot", due to the creation of caches and other optimizations that the underlying operating systems might execute. Raasveldt et al. also alert for "warm" runs, which can occur when the SUT is restarted to perform a "cold" run, but caches are still present in the OS memory [24].

In addition, the authors in [24] suggest practices on how to prevent these pitfalls from happening. In some cases, such as that of non-reproducible experiments, it can be easily avoided by including a detailed specification of the configuration parameters used, hardware specifications and source code. However, other pitfalls are not so trivial to avoid. For instance, making sure the SUTs are not running sub-optimally can be hard depending on the amount of configuration available for each system. Nonetheless, we will follow the checklist provided in [24, Appendix A] and mention it whenever relevant throughout this thesis.

Chapter 3

Related Work

Chapter 4

Methodology

In terms of running time, [14] has a pretty nice approach, where they run each experiment for 15 minutes and repeat it three times. I think I could adopt this approach as well. [13] does 5 minutes per experiment where the first minute is considered "warm up". [1] does runs of three minutes three times. [15] uses runs of 5 minutes.

Chapter 5

Experimental setup

Chapter 6

Results

Chapter 7

Discussion

Chapter 8

Conclusion and Future Work

Bibliography

[1] P. Agnihotri, B. Koldehofe, R. Heinrich, C. Binnig, and M. Luthra, "PDSP-bench: A benchmarking system for parallel and distributed stream processing," en, Accepted at TPCTC, 2024.

[2] Apache Iceberg, *Reliability*, English. Accessed: Oct. 22, 2025. [Online]. Available: https://iceberg.apache.org/docs/latest/reliability/

[3] Apache Iceberg, *REST Catalog Spec*, en, Oct. 2025. Accessed: Oct. 15, 2025. [Online]. Available: https://iceberg.apache.org/rest-catalog-spec/

[4] M. Armbrust et al., "Delta lake: High-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, Aug. 2020, ISSN: 2150-8097. DOI: 10.14778/3415478.3415560 Accessed: Jun. 27, 2025. [Online]. Available: https://dl.acm.org/doi/10.14778/3415478.3415560

[5] Avril Aysha, *Delta Lake vs Data Lake - What's the Difference?* English. Accessed: Jan. 22, 2026. [Online]. Available: https://delta.io/blog/delta-lake-vs-data-lake/

[6] *Ducklake Documentation - Choosing a Catalog Database*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/duckdb/usage/choosing_a_catalog_database

[7] *DuckLake Documentation - Conflict Resolution*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/duckdb/advanced_features/conflict_resolution

[8] *DuckLake Documentation - Data Inlining*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/duckdb/advanced_features/data_inlining

[9] *Ducklake Documentation - Tables*, English, Oct. 2025. Accessed: Jan. 27, 2026. [Online]. Available: https://ducklake.select/docs/stable/specification/tables/overview

[10] M. Fragkoulis, P. Carbone, V. Kalavri, and A. Katsifodimos, "A survey on the evolution of stream processing systems," en, *The VLDB Journal*, vol. 33, no. 2, pp. 507–541, Mar. 2024, ISSN: 1066-8888, 0949-877X. DOI: 10.1007/s00778-023-00819-8 Accessed: Sep. 15, 2025. [Online]. Available: https://link.springer.com/10.1007/s00778-023-00819-8

[11] Guillermo Sanchez and Gabor Szarnyas, *DuckLake 0.3 with Iceberg Interoperability and Geometry Support*, English, Sep. 2025. Accessed: Nov. 21, 2025. [Online]. Available: https://ducklake.select/2025/09/17/ducklake-03/

[12] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," en, *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, Dec. 1983, ISSN: 0360-0300, 1557-7341. DOI: 10.1145/289.291 Accessed: Jan. 22, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/289.291

[13] S. Henning and W. Hasselbring, "Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures," en, *Big Data Research*, vol. 25, p. 100 209, Jul. 2021, ISSN: 22145796. DOI: 10.1016/j.bdr.2021.100209 Accessed: Sep. 15, 2025. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2214579621000265

[14] S. Henning, A. Vogel, M. Leichtfried, O. Ertl, and R. Rabiser, "ShuffleBench: A Benchmark for Large-Scale Data Shuffling Operations with Distributed Stream Processing Frameworks," en, in *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, London United Kingdom: ACM, May 2024, pp. 2–13, ISBN: 979-8-4007-0444-4. DOI: 10.1145/3629526.3645036 Accessed: Sep. 15, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3629526.3645036

[15] G. Hesse, C. Matthies, M. Perscheid, M. Uflacker, and H. Plattner, "ESPBench: The Enterprise Stream Processing Benchmark," en, in *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, Virtual Event France: ACM, Apr. 2021, pp. 201–212, ISBN: 978-1-4503-8194-9. DOI: 10.1145/3427921.3450242 Accessed: Sep. 15, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3427921.3450242

[16] *How to prevent object overwrites with conditional writes*, English. Accessed: Jan. 26, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/conditional-writes.html

[17] *Iceberg*, Dec. 2025. [Online]. Available: https://github.com/apache/iceberg

[18] *Iceberg Spec - Optimistic Concurrency*, English. Accessed: Jan. 26, 2026. [Online]. Available: https://iceberg.apache.org/spec/?h=concurren#optimistic-concurrency

[19] A. M. Jason Hughes, *Apache Iceberg: The Definitive Guide*, eng. O'Reilly Media, Inc, 2024, OCLC: 1425185829, ISBN: 978-1-0981-4861-4.

[20] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking Distributed Stream Data Processing Systems," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, Paris: IEEE, Apr. 2018, pp. 1507–1518, ISBN: 978-1-5386-5520-7. DOI: 10.1109/ICDE.2018.00169 Accessed: Sep. 15, 2025.

[21] Mark Raasveldt and Hannes Mühleisen. "The DuckLake manifesto: SQL as a lakehouse format," DuckLake, Accessed: Jun. 27, 2025. [Online]. Available: https://ducklake.select/manifesto/

[22] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," presented at the CIDR, 2021. Accessed: Jun. 26, 2025. [Online]. Available: https://vldb.org/cidrdb/2021/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics.html

[23] Paras Jain, Peter Kraft, Conor Power, Tathagata Das, Ion Stoica, and Matei Zaharia, "Analyzing and comparing lakehouse storage systems," presented at the CIDR, Amsterdam, The Netherlands, 2023. [Online]. Available: https://vldb.org/cidrdb/2023/analyzing-and-comparing-lakehouse-storage-systems.html

[24] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, "Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing," en, in *Proceedings of the Workshop on Testing Database Systems*, Houston TX USA: ACM, Jun. 2018, pp. 1–6, ISBN: 978-1-4503-5826-2. DOI: 10.1145/3209950.3209955 Accessed: Sep. 8, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3209950.3209955

[25] *Request preconditions*, English, Jan. 2026. Accessed: Jan. 26, 2026. [Online]. Available: https://docs.cloud.google.com/storage/docs/request-preconditions

[26] J. Schneider, C. Gröger, A. Lutsch, H. Schwarz, and B. Mitschang, "The lakehouse: State of the art on concepts and technologies," *SN Computer Science*, vol. 5, no. 5, p. 449, Apr. 18, 2024, ISSN: 2661-8907. DOI: 10.1007/s42979-024-02737-0 Accessed: Jun. 23, 2025. [Online]. Available: https://link.springer.com/10.1007/s42979-024-02737-0

[27] *Specifying conditional headers for Blob service operations*, English, Nov. 2025. Accessed: Jan. 26, 2026. [Online]. Available: https://learn.microsoft.com/en-us/rest/api/storageservices/specifying-conditional-headers-for-blob-service-operations

[28] J. Tahir, R. Mayer, C. Doblander, and H.-A. Jacobsen, "How Reliable are Streams? End-to-End Processing-Guarantee Validation and Performance Benchmarking of Stream Processing Systems," *Proceedings of the VLDB Endowment*, vol. 18, no. 3, pp. 585–598, Nov. 2024, ISSN: 2150-8097. DOI: 10.14778/3712221.3712227 Accessed: Sep. 18, 2025.

[29] Tathagata Das and Denny Lee, *Delta 2.0 - The Foundation of your Data Lakehouse is Open*, English, Aug. 2022. Accessed: Oct. 22, 2025. [Online]. Available: https://delta.io/blog/2022-08-02-delta-2-0-the-foundation-of-your-data-lake-is-open/

[30] The Delta Lake Project Authors, *Delta Lake*, Jan. 2026. Accessed: Jan. 23, 2026. [Online]. Available: https://github.com/delta-io/delta

[31] Tyler Akidau, *Streaming 101: The world beyond batch*, English, Aug. 2015. Accessed: Nov. 12, 2025. [Online]. Available: https://www.oreilly.com/radar/the-world-beyond-batch-streaming-101/

[32] *Unity Catalog Structure*, English, Jul. 2024. Accessed: Jan. 26, 2026. [Online]. Available: https://docs.unitycatalog.io/quickstart/

[33] *What is a data lake?* English. Accessed: Jan. 22, 2026. [Online]. Available: https://www.cloudflare.com/learning/cloud/what-is-a-data-lake/

Appendix A

Dataset details

An appendix, if you need one.